

LINUX SHELLS

[Kaynak](#)

İÇERİK

1. Shell ile Etkileşim Kurma (How To Interact With a Shell?)
2. Linux Shell Türleri (Types of Linux Shells)
3. Shell Scripting ve Bileşenleri (Shell Scripting and Components)
4. Uygulamalı Örnek: Kasa Kilitleme Script'i (The Locker Script)
5. Pratik Uygulama: Log Dosyaları İçinde Flag Arama (Practical Exercise)

Shell ile Etkileşim Kurma (How To Interact With a Shell?)

Bir siber güvenlik araştırmacısı veya sızma testi uzmanı için shell (kabuk), sistemle kurulan en temel ve en güçlü bağıdır. Bu notlarda, bir makineye erişim sağladıktan sonra karşımıza çıkan o siyah ekranda nasıl hareket edeceğimizi, dosyalar arasında nasıl iz süreceğimizi ve veriyi nasıl manipüle edeceğimizi en temelden başlayarak inceleyeceğiz.

1. Sisteme Erişim ve İlk Karşılaşma

Hedef makineye eriştikten sonra bizi karşılayan yapıya **Shell** (Kabuk) diyoruz. İşletim sistemi çekirdeği (kernel) ile kullanıcı arasında bir köprü görevi görür. Linux dünyasında en yaygın kullanılan kabuk **Bash** (Bourne Again Shell) olsa da, dağıtıma göre farklılıklar gösterebilir.

Sisteme bağlanmak için genellikle **SSH** (Secure Shell - Güvenli Kabuk) kullanılır. Eğer elimizde kullanıcı adı ve parola varsa (örneğin `user:user@Tryhackme`), uzak terminale şu şekilde bağlarız:

Bash

```
ssh user@MACHINE_IP
```

Bağlantı sağlandığında karşımıza çıkan `user@tryhackme:~$` ifadesi bir **prompt**'tur. Burada `user` o anki kullanıcıyı, `tryhackme` makine adını (hostname), `~` ise kullanıcının ev dizininde olduğumuzu belirtir.

2. Navigasyon ve Mevcut Konumu Belirleme

Shell'e ilk girdiğimizde "neredeyim?" sorusunun cevabı hayati önem taşır. Grafik arayüzde (GUI) klasörleri görsel olarak görsek de, terminalde bunu komutlarla anlamalıyız.

pwd (Print Working Directory)

Bulunduğumuz tam dosya yolunu (path) ekrana yazdırır.

Bash

```
user@tryhackme:~$ pwd
/home/user
```

Analiz: Bu çıktı bize `/home` dizini altındaki `user` klasöründe olduğumuzu söyler. Çoğu Linux sisteminde varsayılan olarak kullanıcı kendi ev dizininde başlar.

cd (Change Directory)

Dizinler arasında geçiş yapmamızı sağlar. Bir sonraki hamlemizi yapacağımız (dosya okuma, exploit çalıştırma vb.) klasöre gitmek için kullanılır.

Bash

```
user@tryhackme:~$ cd Desktop
user@tryhackme:~/Desktop$
```

Analiz: `cd Desktop` komutuyla bir alt dizine geçtik. Prompt üzerindeki işaretin `~` yerine `~/Desktop` olduğuna dikkat et; bu, konumumuzun değiştiğinin görsel bir kanıtıdır. Eğer `cd ..` yazsaydık bir üst dizine geri dönerdik.

3. İçerik Listeleme ve Okuma

Bir klasöre girdikten sonra içeride ne olduğunu görmeden hareket edemeyiz. Bu noktada "gözümüz" **ls** komutudur.

ls (List)

Mevcut dizindeki dosya ve klasörleri listeler.

Bash

```
user@tryhackme:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

Not: Eğer gizli dosyaları (başında nokta olanlar, örn: `.ssh`) görmek isteseydik `ls -a` parametresini kullanmamız gerekirdi. Dosya izinlerini ve boyutlarını görmek için ise `ls -l` (long format) tercih edilir.

cat (Concatenate)

Dosya içeriğini terminal ekranına basmak için kullanılır. Özellikle **Enumeration** (Bilgi Toplama) aşamasında konfigürasyon dosyalarını veya "flag" dosyalarını okumak için en sık kullanılan araçtır.

Bash

```
user@tryhackme:~$ cat filename.txt
this is a sample file
this is the second line of the file
```

Dikkat: Çok büyük dosyalarda `cat` kullanmak ekranı binlerce satırla doldurabilir. Bu gibi durumlarda `less` veya `tail` komutları daha mantıklı bir alternatif olabilir.

4. Dosya İçinde Arama Yapma: `grep`

Sistemde binlerce satırlık log dosyaları veya devasa sözlük dosyaları (wordlist) arasında belirli bir kelimeyi aramak manuel olarak imkansızdır. Burada devreye **grep** (Global Regular Expression Print) girer.

grep, kendisine verilen bir kalıbı (pattern) belirtilen dosya içinde arar ve sadece o kalıbın geçtiği satırları ekrana getirir.

Bash

```
user@tryhackme:~$ grep THM dictionary.txt
The flag is THM
```

Komut Parçalaması:

- **grep**: Arama motorunu çağırır.
- **THM**: Aranacak anahtar kelime (case-sensitive yani büyük-küçük harf duyarlıdır).
- **dictionary.txt**: Aramanın yapılacağı hedef dosya.

Neden Önemli? Örneğin binlerce satırlık bir konfigürasyon dosyasında "password" kelimesini arıyorsanız, `grep "password" config.php` komutu sizi saatlerce sürececek bir okuma zahmetinden kurtarır. Eğer büyük-küçük harf fark etmeksizin arama yapmak isterseniz `-i` parametresini (`grep -i "thm"`) ekleyebilirsiniz.

Linux Shell Türleri (Types of Linux Shells)

Windows'taki Komut İstemi (CMD) ve PowerShell gibi, Linux dünyasında da her biri farklı yeteneklere ve karakteristik özelliklere sahip çeşitli kabuklar (shells) bulunur. Bir siber güvenlik uzmanı için hangi shell içerisinde çalıştığını bilmek, kullanabileceği araçları ve yazabileceği script'lerin uyumluluğunu belirler.

1. Mevcut ve Kullanılabilir Shell'lerin Tespiti

Sistemde o an hangi shell'i kullandığınızı veya hangi shell'lerin yüklü olduğunu bilmek, çevre birimlerini kontrol etmek adına ilk adımdır.

echo \$SHELL

O an aktif olarak kullandığınız kabuğun yolunu ekrana yazdırır. `$SHELL` bir ortam değişkenidir (environment variable).

Bash

```
user@tryhackme:~$ echo $SHELL
/bin/bash
```

/etc/shells Dosyası

Sistemde yüklü olan ve "geçerli" kabul edilen tüm kabukların listesi bu dosyada tutulur.

Bash

```
user@tryhackme:~$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/usr/bin/zsh
/usr/bin/tmux
/bin/rbash
```

Analiz: Burada gördüğünüz **rbash** (Restricted Bash), kısıtlı bir kabuktur ve genellikle sızma testlerinde "Escape" (kaçış) yapmanız gereken bir engel olarak karşınıza çıkar. **tmux** ve **screen** ise aslında terminal çoklayıcılarıdır ancak burada kabuk olarak tanımlanmışlardır.

2. Shell Değiştirme ve Varsayılan Yapma

Sistemde yüklü olan başka bir kabuğa geçmek oldukça basittir.

- **Anlık Geçiş:** Sadece kabuğun adını yazmanız yeterlidir. Örneğin `zsh` yazıp Enter'a bastığınızda prompt değişecektir: Bash

```
user@tryhackme:~$ zsh
tryhackme%
```

- **Kalıcı Değişim (chsh):** Varsayılan kabuğunuzu kalıcı olarak değiştirmek için `chsh` (change shell) komutu kullanılır. Bash

```
chsh -s /usr/bin/zsh
```

Mantık: `-s` (shell) parametresi, kullanıcı profilinizdeki varsayılan kabuk yolunu günceller. Bir sonraki oturum açışınızda artık otomatik olarak Zsh ile başlarsınız.

3. En Yaygın Linux Shell'leri ve Özellikleri

A. Bash (Bourne Again Shell)

Linux dünyasının "endüstri standardı"dır. Çoğu dağıtımda varsayılan olarak gelir. Eski kabukların (sh, ksh, csh) en iyi özelliklerini bir araya getirmiştir.

- **Tab Completion:** Komut yazarken TAB tuşuna basarsanız komutu tamamlar veya seçenekleri sunar.
- **History:** Yazdığınız tüm komutları bir dosyada tutar. Üst/Alt ok tuşlarıyla eski komutlara erişebilir veya history yazarak tam listeyi görebilirsiniz.
- **Scripting:** Otomatize işler için en geniş dökümantasyona sahip betik dilidir.

B. Fish (Friendly Interactive Shell)

Adından da anlaşılacağı üzere kullanıcı dostu olmayı hedefler. Yeni başlayanlar için harikadır.

- **Otomatik Düzeltme:** Yazım hatalarını fark eder ve düzeltme önerir.
- **Syntax Highlighting:** Komutun parametrelerini, dosya yollarını farklı renklerde boyayarak okunabilirliği artırır.
- **Akıllı Öneriler:** Siz yazarken geçmiş komutlarınıza dayanarak gri renkte tamamlamalar sunar.

C. Zsh (Z Shell)

Bash'in modern ve "steroid almış" versiyonu gibidir. Özellikle **Kali Linux** gibi dağıtımların varsayılan kabuğudur.

- **Gelişmiş Özelleştirme:** oh-my-zsh gibi framework'ler ile inanılmaz görsel ve fonksiyonel eklentiler eklenebilir.
- **Eklenti Desteği:** Bash'ten çok daha yetenekli bir tab tamamlama ve otomatik düzeltme mekanizmasına sahiptir.

4. Karşılaştırmalı Shell Tablosu

Aşağıdaki tablo, sızma testi veya sistem yönetimi sırasında hangi kabuğu seçeceğinize karar vermenize yardımcı olur:

Özellik	Bash	Fish	Zsh
Tam Adı	Bourne Again Shell	Friendly Interactive Shell	Z Shell
Scripting	Çok güçlü ve standart.	Kısıtlı / Farklı senaryo.	Çok güçlü ve modern.

Özellik	Bash	Fish	Zsh
Tab Tamamlama	Temel seviye.	Gelişmiş (Geçmişe dayalı).	Çok gelişmiş (Eklentilerle).
Özelleştirme	Temel seviye.	İnteraktif araçlarla kolay.	Çok ileri (oh-my-zsh).
Sözdizimi Boyama	Yok (Eklentisiz).	Yerleşik (Built-in).	Eklentilerle mümkün.

Kritik Not: Bir siber güvenlikçi olarak her zaman en azından temel **Bash** bilgisine sahip olmalısınız; çünkü sızdığınız makinelerin %99'unda Fish veya Zsh değil, Bash veya Sh sizi karşılayacaktır.

Shell Scripting ve Bileşenleri (Shell Scripting and Components)

Shell scripting, terminalde tek tek yazdığımız komutları bir dosyaya toplayıp otomatize etme sanatıdır. Tekrarlayan görevleri her seferinde manuel yapmak yerine, bir script hazırlayıp saniyeler içinde çalıştırmak hem zaman kazandırır hem de hata payını düşürür.

1. Script Oluşturma ve Hazırlık

Bir shell script'i temelde `.sh` uzantılı bir metin dosyasıdır. Yazmaya başlamak için şu adımları izleriz:

Dosya Oluşturma ve Nano Editor

Terminalde `nano first_script.sh` komutuyla düzenleyiciyi açarız.

Shebang (#!)

Her script'in en başında, bu dosyanın hangi yorumlayıcı (interpreter) tarafından çalıştırılacağını belirten bir satır bulunmalıdır. Buna **Shebang** denir. Bash script'leri için standart şudur:

Bash

```
#!/bin/bash
```

Neden Önemli? Eğer bu satırı eklemesiniz, sistem dosyayı hangi dille (sh, bash, python, zsh vb.) okuyacağını bilemeyebilir ve script hatalı çalışabilir.

Çalıştırma Yetkisi (chmod)

Linux'ta yeni oluşturulan dosyalar varsayılan olarak "çalıştırılabilir" değildir. Script'i çalıştırmadan önce şu komutu vermeliyiz:

Bash

```
chmod +x first_script.sh
```

- `chmod` : Dosya izinlerini değiştirir.
- `+x` : Çalıştırma (execute) yetkisi ekler.

Script'i Başlatma (./)

Script'i çalıştırmak için başına `./` ekleriz:

Bash

```
./first_script.sh
```

Mantık: Eğer sadece `first_script.sh` yazarsanız, sistem bu dosyayı global **PATH** dizinlerinde (`bin`, `sbin` vb.) arar ve bulamazsa hata verir. `./` ifadesi kabuğa "şu an içinde bulunduğum dizine bak" talimatını verir.

2. Temel Yapı Taşları

A. Değişkenler (Variables)

Değişkenler, bir değeri (IP adresi, dosya yolu, kullanıcı adı vb.) hafızada tutup tekrar tekrar kullanmamızı sağlar.

Bash

```
#!/bin/bash
echo "Hey, adın ne?"
read name # Kullanıcıdan girdi bekler ve 'name' değişkenine atar.
echo "Hoş geldin, $name" # Değişkeni çağırırken başına '$' koyarız.
```

Analiz: `read` komutu interaktif bir etkileşim sağlar. Değişkeni ekrana yazdırırken `$name` şeklinde çağırarak, kabuğa "bu ismin içindeki değeri getir" demektir.

B. Döngüler (Loops)

Bir işlemi birden fazla kez tekrar etmek için kullanılır. Örneğin, 1'den 10'a kadar sayıları yazdırmak:

Bash

```
#!/bin/bash
for i in {1..10}; do
```

```
echo $i
done
```

Komut Parçalaması:

- `for i in {1..10} :` `i` değişkeni sırayla 1, 2, 3... 10 değerlerini alır.
- `do :` Döngü bloğunun başladığını belirtir.
- `done :` Döngünün bittiğini belirtir.
- İçerideki `echo $i` her adımda o anki sayıyı ekrana basar.

C. Koşullu İfadeler (Conditional Statements)

Belirli bir durumun doğru olup olmadığını kontrol eder. Bir sızma testi script'inde "Eğer port açıksa şunu yap, kapalıysa bunu yap" demek için kullanılır.

Bash

```
#!/bin/bash
echo "İsmini gir:"
read name

if [ "$name" = "Stewart" ]; then
    echo "Giriş başarılı! Gizli kod: THM_Script"
else
    echo "Yetkisiz kullanıcı!"
fi
```

Dikkat Edilecek Noktalar:

- `if [ condition ] :` Köşeli parantezler ile içerik arasında mutlaka **boşluk** olmalıdır.
- `then :` Koşul doğruysa yapılacak işlemi başlatır.
- `else :` Koşul yanlışsa devreye girer.
- `fi :` `if` bloğunu kapatır (`if` kelimesinin tersidir).

3. Kodun Okunabilirliği: Yorum Satırları (Comments)

Kod büyüdükçe neyin ne işe yaradığını unutmamak için `#` işareti ile yorumlar eklenir. Bash, `#` ile başlayan satırları çalıştırmaz, görmezden gelir.

Bash

```
#!/bin/bash
# Bu bir yorumdur, terminalde gözükmez.
echo "Bu ise ekrana yazılır."
```

Özet Tablo: Script Bileşenleri

Bileşen	Örnek Kullanım	Amacı
Shebang	<code>#!/bin/bash</code>	Script'in hangi kabukla çalışacağını belirler.
Variables	<code>ip="10.10.10.10"</code>	Veriyi isimli bir etikette saklar.
read	<code>read target</code>	Kullanıcıdan klavye girişi alır.
chmod +x	<code>chmod +x dosya.sh</code>	Dosyayı bir program gibi çalışabilir hale getirir.
if/else	<code>if [\$a == \$b]</code>	Mantıksal kararlar verir.
for/do/done	<code>for x in list</code>	Belirli bir işlemi tekrarlar.

Uygulamalı Örnek: Kasa Kilitleme Script'i (The Locker Script)

Şimdiye kadar öğrendiğimiz tüm bileşenleri (değişkenler, döngüler ve koşullu ifadeler) tek bir senaryoda birleştirelim. Amacımız, bir banka kasasına erişim sağlamak isteyen kullanıcıyı doğrulayan profesyonel bir script yazmak.

1. Senaryo Gereksinimleri

Script çalıştığında kullanıcıdan sırasıyla üç bilgi istenmeli:

- Kullanıcı Adı:** John
- Şirket Adı:** Tryhackme
- PIN Kodu:** 7385

Eğer bu üç bilgi de doğru girilirse erişim verilecek, aksi takdirde reddedilecektir.

2. Script'in Kod Yapısı ve Analizi

Önce kodu bloklar halinde inceleyelim, ardından mantık akışını (logic flow) çözelim.

Bash

```
#!/bin/bash
```

```
# --- 1. Bölüm: Değişkenlerin Tanımlanması ---
```

```
# Başlangıçta değişkenleri boş olarak tanımlıyoruz.
```

```
username=""
```

```
companyname=""
```

```
pin=""
```

```
# --- 2. Bölüm: Döngü ve Veri Toplama ---
```

```
# Toplam 3 giriş alacağımız için 1'den 3'e kadar dönen bir döngü kuruyoruz.
```

```
for i in {1..3}; do
```

```
    # 'i' değişkeni o an kaçınıcı adımda olduğumuzu tutar.
```

```
    if [ "$i" -eq 1 ]; then
```

```
    echo "Enter your Username:"
    read username
elif [ "$i" -eq 2 ]; then
    echo "Enter your Company name:"
    read companyname
else
    echo "Enter your PIN:"
    read pin
fi
done

# --- 3. Bölüm: Mantıksal Doğrulama ---
# '&&' (AND) operatörü ile tüm şartların aynı anda sağlanıp sağlanmadığına
bakıyoruz.
if [ "$username" = "John" ] && [ "$companyname" = "Tryhackme" ] && [ "$pin"
= "7385" ]; then
    echo "Authentication Successful. You can now access your locker, John."
else
    echo "Authentication Denied!!"
fi
```

Kodun Teknik Detayları:

- `for i in {1..3}; do` : Burada döngü bir sayaç görevi görüyor. 1. adımda kullanıcı adını, 2. adımda şirket adını, 3. adımda PIN kodunu sormak için program akışını yönetiyor.
- `-eq` **Parametresi**: "Equal" (eşit) kelimesinin kısaltmasıdır. Sayısal karşılaştırmalar için kullanılır. `$i -eq 1` demek, "Eğer `i` sayısı 1'e eşitse" demektir.
- `elif` (**Else If**): İlk şart sağlanmazsa ikinci bir şartı kontrol eder.
- `&&` (**Mantıksal VE**): Koşullu ifadenin (if) başarılı sayılması için her üç değişkenin de (`username` , `companyname` , `pin`) doğru olması gerektiğini belirtir. Biri bile yanlış olsa sonuç `else` bloğuna düşer.

3. Script'in Test Edilmesi (Execution)

Script'i `nano locker_script.sh` ile kaydedip `chmod +x locker_script.sh` yetkisini verdikten sonra çalıştırdığımızda şu çıktıyı alırız:

Hatalı Giriş Durumu:

Plaintext

```
user@tryhackme:~$ ./locker_script.sh
Enter your Username:
John
Enter your Company name:
Tryhackme
```

Enter your PIN:

1349

Authentication Denied!!

Analiz: Kullanıcı adı ve şirket doğru olsa bile, PIN kodu (1349) sistemdeki kayıtlı (7385) eşleşmediği için erişim engellendi. Bu, güvenlik mekanizmasının (&&) doğru çalıştığını kanıtlar.

Önemli Notlar ve İpuçları

- Güvenlik Uyarısı:** Gerçek dünyada PIN veya şifre gibi hassas bilgiler script içerisine "hardcoded" (statik olarak yazılarak) gömülmez. Bu bilgiler genellikle şifrelenmiş veritabanlarından veya güvenli kasalardan çekilir.
- Hata Payı:** Script'te `read` komutuyla alınan verilerde büyük/küçük harf duyarlılığı (case-sensitivity) vardır. "john" girilirse doğrulama başarısız olacaktır.

Pratik Uygulama: Log Dosyaları İçinde Flag Arama (Practical Exercise)

Bu görevde, öğrendiğimiz tüm Shell yeteneklerini gerçek bir senaryoda test ediyoruz. Elimizde bir dizin dolusu `.log` dosyası var ve bizim amacımız, bir script yardımıyla bu dosyaların içinde gizlenmiş olan spesifik bir anahtar kelimeyi (**flag**) bulmak.

1. Hazırlık: Root Yetkisi Almak

Sistem logları genellikle `/var/log` dizininde tutulur ve bu dizindeki dosyalara erişmek için yüksek yetki (root) gerekebilir. Eğer normal kullanıcı (`user`) olarak arama yaparsak, "Permission Denied" (Erişim Engellendi) hatalarıyla karşılaşabiliriz.

sudo su (Switch User)

Root kullanıcısına geçmek için şu komutu kullanırız:

Bash

```
user@tryhackme:~$ sudo su
[sudo] password for user: # Buraya 'user' parolasını giriyoruz
root@tryhackme:/home/user#
```

Analiz: Prompt kısmının `$` işaretinden `#` işaretine dönüştüğünü göreceksin. Bu, artık sistemde en yetkili kullanıcı olduğun anlamına gelir.

2. Script'i Düzenleme (The Missing Pieces)

/home/user dizininde bulunan script, içinde boş bırakılmış tırnak işaretleri (" ") içeriyor. Bu boşlukları hedeflerimize göre doldurmamız gerekiyor.

Senaryo Bilgileri:

- **Aranacak Kelime (Flag):** thm-flag01-script
- **Aranacak Dizin:** /var/log

Script'in İçeriği ve Doldurulması

nano search_script.sh (veya dosya adı neyse) diyerek dosyayı açıyoruz ve şu mantıkla düzenliyoruz:

Bash

```
#!/bin/bash

# Değişkenleri senaryoya göre dolduruyoruz
DIRECTORY="/var/log"
KEYWORD="thm-flag01-script"

# Script'in geri kalanı muhtemelen şöyle bir mantıkla çalışıyor:
for file in $DIRECTORY/*.log; do
    grep "$KEYWORD" "$file"
done
```

Kritik Uyarı: Tırnak işaretlerinin içini doldururken arada boşluk bırakmadığından emin ol (" /var/log" yerine "/var/log").

3. Script'i Çalıştırma ve Sonuç

Script'i düzenleyip kaydettikten (CTRL+O , ENTER , CTRL+X) sonra çalıştırma yetkisi verip başlatıyoruz:

Bash

```
root@tryhackme:/home/user# chmod +x search_script.sh
root@tryhackme:/home/user# ./search_script.sh
```

Neler Oluyor?

1. **Döngü Başlar:** Script /var/log altındaki tüm .log dosyalarını tek tek sıraya koyar.
2. **Arama Yapılır:** grep komutu her bir dosyanın içinde thm-flag01-script kelimesini arar.
3. **Çıktı:** Eğer kelime bir dosyanın içinde bulunursa, o satır ekrana basılır ve flag karşımıza çıkar.

Özet ve Mantık Zinciri

- **Neden Root olduk?** Okuma yetkimizin olmadığı log dosyalarını atlamamak için.
- **Neden Script kullandık?** `/var/log` içinde onlarca dosya olabilir; her birine tek tek `grep` atmak yerine bir döngü ile işi saniyeler içinde bitirdik.
- **Değişkenler ne işe yaradı?** Script'in başında `DIRECTORY` ve `KEYWORD` tanımlayarak kodu daha sonra başka aramalar için de kullanılabilir (reusable) hale getirdik.